MENTORIA DE IA APLICADA · APOSTILA

# Módulo 07 — Human-in-the-loop e automação

Por que e onde pôr uma pessoa no circuito, níveis de autonomia, interrupt() e Command, as armadilhas da retomada e como medir a revisão.

---


1. Visão geral
2. Conceitos
3. Exercício
4. Projeto final — etapa M7
5. Para revisar


Fonte: https://github.com/MateusCastro2203/Mentoria-AI

# Visão geral

## Objetivos de aprendizagem

Ao final, a pessoa consegue:

1. explicar por que e onde pôr uma pessoa no circuito, incluindo o risco do viés de automação;
2. classificar uma etapa num nível de autonomia (operadora → observadora) e justificar a escolha;
3. pausar um grafo com `interrupt()` e retomá-lo com `Command({ resume })`, inclusive com várias pausas em paralelo, aprovando, editando ou rejeitando;
4. evitar as armadilhas da retomada (o nó reexecuta desde o começo; nada de efeito externo antes da pausa);
5. medir um processo com revisão humana (taxa de intervenção, taxa de override, tempo de decisão) e calibrar o limiar de automação com dados.

## Mensagem central

> Automatize o que dá para desfazer; peça aprovação para o que não dá. Comece com mais revisão e vá soltando **com dados**. E meça se a revisão é real: uma pessoa que aprova tudo em dois segundos virou carimbo.

# Conceitos

No Módulo 6, o grafo do curador mandava os itens incertos para uma `fila_humana`, que só **anotava** os itens e seguia em frente. Neste módulo, a fila vira uma **pausa de verdade**: o grafo para, guarda o estado e espera uma pessoa decidir, por minutos ou por dias. A pergunta de fundo é: **o que a máquina pode fazer sozinha, e o que precisa de alguém?**

---

## 1. Por que humano no circuito

*Human-in-the-loop* (HITL, "humano no circuito") é desenhar o sistema para que uma pessoa revise, corrija ou autorize partes do trabalho. Três motivos:

1. **Ações que não voltam atrás.** Publicar para milhares de leitores, pagar um reembolso, apagar dados. Um erro aí custa mais que o tempo de uma revisão.
2. **Incerteza.** O modelo sabe quando está incerto? Só se a confiança for **calibrada** (Aula 2). Quando é, ela serve de gatilho: confiança baixa → uma pessoa olha.
3. **Responsabilidade.** Algumas decisões precisam ter alguém que responda por elas: negar um pedido de um cliente, aprovar crédito, publicar em nome da empresa.

**Analogia:** o piloto automático do avião. Ele voa sozinho a maior parte do tempo, mas a decolagem, o pouso e as situações fora do normal ficam com os pilotos. E os pilotos são treinados para **não** confiar cegamente nele.

### O risco do outro lado: viés de automação

Pôr uma pessoa no circuito não resolve tudo. Parasuraman e Manzey (2010) revisaram décadas de estudos sobre como pessoas usam sistemas automáticos e descrevem o **viés de automação**: a tendência de aceitar a sugestão da máquina e de deixar de procurar sinais contrários. Ele aparece em iniciantes e em especialistas, em pessoas sozinhas e em equipes, e não some com treinamento ou instruções.

Consequências para o desenho:

- **Não peça aprovação de tudo.** Revisar 100 itens por dia vira carimbo; a atenção vai para onde importa quando a pausa é rara.
- **Mostre o motivo da pausa e a evidência** (por que este item parou? qual a confiança? qual a fonte?), não só "aprova?".
- **Meça se a revisão está funcionando** (seção 5): uma revisão que nunca discorda pode ser uma revisão que não está olhando.

---

## 2. Níveis de autonomia

Feng, McDonald e Zhang (2025) propõem cinco níveis de autonomia para agentes, definidos pelo **papel da pessoa**:

| Nível | Papel da pessoa | Em uma frase |
|---|---|---|
| 1 | **Operadora** (*operator*) | a pessoa comanda o tempo todo; o agente ajuda quando é chamado |
| 2 | **Colaboradora** (*collaborator*) | pessoa e agente planejam, delegam e executam juntos |
| 3 | **Consultora** (*consultant*) | o agente toma a iniciativa e consulta a pessoa nas decisões importantes |
| 4 | **Aprovadora** (*approver*) | o agente trabalha sozinho e só chama a pessoa num bloqueio, ou para aprovar uma ação de consequência |
| 5 | **Observadora** (*observer*) | totalmente autônomo: a pessoa só acompanha pelos registros (e pode desligar) |

O ponto central do artigo: **o nível de autonomia é uma decisão de desenho**, separada da capacidade do modelo. Um modelo muito capaz pode (e às vezes deve) operar no nível 4.

No curso:

- **Curador:** os itens incertos ficam no nível 4 (aprovadora); a edição inteira também, antes de publicar. Com `revisarEdicao: false`, a publicação sobe para o nível 5.
- **Reembolso (exercício):** a política escolhe o nível **por pedido**: pequeno e confiante → 5 (paga sozinho); médio ou incerto → 4 (aprovação); grande demais → o agente nem age (escala).

---

## 3. A mecânica: `interrupt()` e `Command({ resume })`

O LangGraph.js pausa um grafo com `interrupt()`, dentro de um nó:

```
.addNode("pedir_aprovacao", (estado) => {
  const resposta = interrupt({ pedido: estado.pedido, analise: estado.analise }); // ⏸ pausa aqui
  // … quando retomar, `resposta` é o que a pessoa mandou
})
```

1. O grafo **salva o estado** no checkpointer (Aula 6) e devolve o controle. O valor passado ao `interrupt()` aparece no resultado, em `__interrupt__`, para a aplicação mostrar à pessoa.
2. Para continuar, invoque de novo **na mesma thread** com um `Command`:

```
await grafo.invoke(new Command({ resume: { acao: "aprovar" } }), { configurable: { thread_id } });
```

O valor de `resume` vira o **retorno** do `interrupt()`.

3. Como a pausa está no checkpoint, a resposta pode vir de **outro processo**, horas depois: um terminal, uma página web, um bot de chat.

## Várias pausas ao mesmo tempo

Se vários nós em paralelo pausam (no curador, um `revisao_humana` por item incerto), cada pausa tem um `id`. Responda com um mapa `id → resposta`:

```
const respostas = Object.fromEntries(resultado.__interrupt__.map((p) => [p.id, decidir(p.value)]));
await grafo.invoke(new Command({ resume: respostas }), config);
```

Dá para responder só algumas: as outras continuam pausadas.

## As regras que mordem

A documentação do LangGraph lista regras que parecem detalhe, mas causam bugs:

1. **Ao retomar, o nó roda de novo desde o começo.** Tudo o que vem antes do `interrupt()` acontece outra vez. Se ali tiver um e-mail, uma cobrança ou uma chamada cara de LLM, ela se repete (`demo:armadilha`). Regra prática: **efeitos externos depois da pausa, de preferência num nó separado**. No nosso teste, responder uma de três pausas em paralelo fez as três reexecutarem até o `interrupt()`.
2. **Não envolva `interrupt()` em `try/catch`.** Ele funciona lançando uma exceção especial; o `catch` engole a pausa.
3. **A ordem importa.** Se um nó tem mais de um `interrupt()`, as respostas são casadas pela posição. Não pule pausas condicionalmente entre uma execução e outra.
4. **O valor da pausa e a resposta precisam caber em JSON**, porque vão para o checkpoint.

## Aprovar, editar, rejeitar

Os três padrões mais comuns de resposta:

| Resposta | O que faz | No curador | No reembolso |
|---|---|---|---|
| **aprovar** | segue como a máquina propôs | o item vai para a redação | paga o valor sugerido |
| **editar** | segue com uma correção | outra categoria ou título | paga outro valor |
| **rejeitar** | não segue; registra o motivo | o item sai | nega o reembolso |

**Valide a resposta humana com schema**, como a saída de um LLM (Aula 3). Uma resposta fora do formato não pode virar "aprovado" por acidente. No exercício, uma resposta inválida faz o grafo **perguntar de novo**, mostrando o erro.

---

## 4. O que automatizar 100%

Não há regra universal, mas três perguntas ajudam a escolher o nível de cada etapa:

1. **Dá para desfazer?** Rascunho, classificação interna, sugestão: sim → pode ser automático. Pagamento, publicação, mensagem ao cliente: não → peça aprovação até ter dados que justifiquem o contrário.
2. **Quanto custa um erro?** Um item fora do tema na newsletter custa pouco; um reembolso de R$ 5.000 errado custa muito.
3. **A confiança é calibrada e há dados?** Se, no histórico, os itens com confiança ≥ 0,95 acertaram 99% das vezes, automatizar essa faixa é uma decisão baseada em evidência. Sem histórico, comece com mais revisão e vá soltando.

A autonomia não é fixa: **comece no nível 4 e suba com dados**. Cada decisão humana registrada vira histórico para recalibrar o limiar (é o desafio do exercício) e vira caso de avaliação (Aula 3).

---

## 5. Como medir

Três números simples dizem se o HITL está funcionando:

| Métrica | Conta | O que indica |
|---|---|---|
| **Taxa de intervenção** | itens que precisaram de uma pessoa ÷ itens processados | quanto do trabalho ainda depende de gente (custo) |
| **Taxa de override** | decisões em que a pessoa editou ou rejeitou ÷ decisões | quanto a pessoa discorda da máquina (qualidade da máquina, e se a revisão é real) |
| **Tempo de decisão** | quanto a pessoa leva para decidir | o custo de cada pausa e a qualidade da tela de revisão |

Leitura conjunta:

- **Intervenção alta + override baixo:** a máquina acerta e a pessoa só carimba. Hora de subir o limiar de automação, com dados.
- **Override alto:** a máquina erra muito no que manda para revisão. Ótimo que a pessoa está lá; melhore o modelo ou a política.
- **Override quase zero + decisões em poucos segundos:** cuidado com o viés de automação. Teste a revisão de vez em quando com um caso que **deveria** ser rejeitado.

### Calibrar o limiar com dados

O limiar de confiança (0,9 no curador) começa como um chute. Com histórico, ele vira uma escolha:

- para cada limiar possível, calcule a **cobertura** (quanto seria automático) e a **precisão** (quanto disso estaria certo);
- escolha o **menor limiar** que ainda atinge a precisão que o negócio exige.

A precisão **não** cai de forma regular conforme o limiar desce: uma faixa de confiança pode ter mais erros que a de baixo. Por isso, olhe a curva inteira.

---

## 6. A tela de revisão (leitura extra)

O melhor `interrupt()` do mundo não ajuda se a pessoa não tem o que precisa para decidir. Uma boa tela de revisão:

- diz **por que** o item parou (confiança baixa? guardrail? qual?);
- mostra a **evidência** (fonte, trecho, confiança) ao lado da proposta;
- tem ações claras (aprovar, editar, rejeitar) e pede o **motivo** de uma rejeição (vira dado);
- agrupa o que dá para decidir em lote, mas sem esconder os casos difíceis.

No curso, a "tela" é o terminal (`m07:edicao`). Trocar por uma página web ou um bot de chat não muda o grafo: só quem chama `Command({ resume })`.

---

## Referências

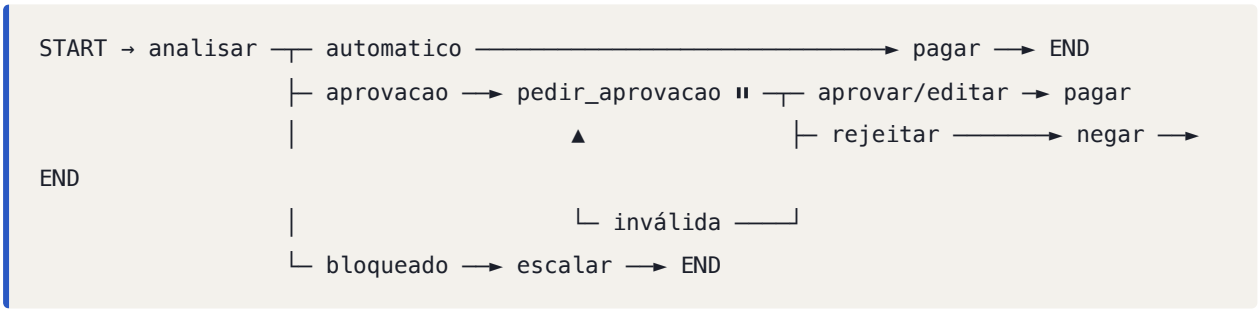

- LangGraph.js — interrupts: https://docs.langchain.com/oss/javascript/langgraph/interrupts
- LangGraph.js — human-in-the-loop: https://docs.langchain.com/oss/javascript/langgraph/human-in-the-loop
- Feng, K. J. K., McDonald, D. W., Zhang, A. X. (2025). *Levels of Autonomy for AI Agents*. arXiv:2506.12469. https://arxiv.org/abs/2506.12469
- Parasuraman, R., Manzey, D. H. (2010). *Complacency and Bias in Human Use of Automation: An Attentional Integration*. Human Factors, 52(3), 381–410. https://doi.org/10.1177/0018720810376055
- Anthropic (2024). *Building effective agents* (pontos de controle humanos em agentes). https://www.anthropic.com/engineering/building-effective-agents

# Exercício

> Tarefa assíncrona. Os testes rodam offline: não precisa de modelo rodando.

Um assistente analisa pedidos de reembolso e sugere pagar ou negar. **Pagar é uma ação no mundo**: o dinheiro sai e não volta. Então nem tudo pode ser automático:

| Nível | Quando | O que acontece |
|---|---|---|
| **automático** | valor pequeno, confiança alta, sugestão a favor do cliente | paga sozinho |
| **aprovação** | o resto (valor médio, confiança baixa, toda sugestão de negar) | o grafo **pausa** e uma pessoa aprova, edita o valor ou rejeita |
| **bloqueado** | valor acima do limite | escala para outra equipe, sem pagar |




## Como rodar

Da raiz do repositório:

```
pnpm -F @mentoria/ex07-hitl test             # versão base
pnpm -F @mentoria/ex07-hitl test:desafio     # desafio extra
pnpm -F @mentoria/ex07-hitl demo:aprovacao   # três pedidos; no do meio, você decide no
terminal
pnpm -F @mentoria/ex07-hitl demo:armadilha   # por que nada de efeito antes do
interrupt()
```

Os testes **falham** até você implementar. A solução de referência está em `solucao/`. Tente antes de abrir. `pnpm -F @mentoria/ex07-hitl test:solucao` roda os mesmos testes contra ela.

## Versão base

| Arquivo | O que fazer |
|---|---|
| `src/autonomia.ts` | `nivelDeAutonomia`: a política (valor, confiança, sugestão → nível) |
| `src/reembolso.ts` | `montarReembolso`: o grafo, com `interrupt()` em `pedir_aprovacao` |

Fornecidos em `src/estado.ts`: o estado, os formatos (`PedidoSchema`, `AnaliseSchema`, `RespostaSchema`) e a política padrão.

**Dicas:**

- `interrupt(valor)` pausa o grafo e entrega `valor` a quem chamou, no campo `__interrupt__` do resultado. Para continuar: `grafo.invoke(new Command({ resume: resposta }), config)`, com o mesmo `thread_id`. A `resposta` vira o retorno do `interrupt()`.
- Ao retomar, **o nó roda de novo desde o começo**. Tudo o que vem antes do `interrupt()` acontece outra vez. Por isso `pagar` é um nó separado, depois da decisão. Rode `demo:armadilha` para ver.
- Não ponha `interrupt()` dentro de `try/catch`: ele funciona lançando uma exceção especial, e o `catch` engoliria a pausa.
- Valide a resposta humana como valida a saída de um LLM: com schema. Uma resposta inválida não pode virar "aprovado".

**Feito =** `pnpm -F @mentoria/ex07-hitl test` com todos os testes passando.

## Desafio extra — calibrar o limiar com dados

O limite de confiança 0,9 foi um chute. Em `src/desafio/limiar.ts`, use o histórico (confiança do assistente × a sugestão estava certa?) para:

1. `curvaDeLimiar`: para cada limiar possível, quanto seria automatizado (**cobertura**) e quanto acertaria (**precisão**);
2. `escolherLimiar`: o menor limiar que ainda atinge a precisão pedida, ou seja, o máximo de automação com a qualidade exigida.

É a ponte com a Aula 2: só dá para usar a confiança como gatilho se ela for **calibrada**.

**Feito =** `pnpm -F @mentoria/ex07-hitl test:desafio` com todos os testes passando.

## Perguntas para pensar

1. Na sua política, por que **toda** sugestão de negar passa por uma pessoa, mesmo com confiança alta?

2. O que aconteceria se `pagar` fosse chamado dentro de `pedir_aprovacao`, antes do `interrupt()`?
3. Se a pessoa aprova 99% do que vê sem mudar nada, o que isso diz? O processo está bom ou a revisão virou carimbo?
4. No seu trabalho, que decisão você deixaria 100% automática, qual pediria aprovação e qual nunca deixaria com uma máquina?

# Projeto final — etapa M7

## Etapa M7 — humanos no circuito

**Objetivo:** a `fila_humana` da M6 vira uma **pausa de verdade**. O grafo para em cada item incerto e, antes de publicar, na edição inteira; uma pessoa aprova, edita ou rejeita pelo terminal, quando puder (a pausa fica salva no checkpoint).




Implemente em `src/m07/`:

1. **`decisoes.ts`:** funções puras:
   - `aplicarDecisaoItem`: aprovar, editar (categoria, título) ou rejeitar um item, validando a resposta com schema;
   - `aplicarDecisaoEdicao`: aprovar, editar (remover itens, reescrever resumos) ou rejeitar a edição;
   - `metricasDeRevisao`: pausas, taxa de intervenção, taxa de override e tempo médio de decisão.
2. **`grafo.ts`:** `montarGrafoHitl`, o grafo da M6 com os nós `revisao_humana` (um `interrupt()` por item, **antes** de qualquer chamada ao modelo) e `aprovar_edicao`.

Fornecidos: `estado.ts` (estado, formatos de pergunta e resposta, `pausasPendentes`) e `redacao.ts` (o ciclo redator ⇄ revisor como função).

**Feito =**

1. `pnpm -F @mentoria/curador exec vitest run test/m07.test.ts` verde (offline);
2. uma edição revisada por você e publicada:

   ```
   pnpm -F @mentoria/curador m07:edicao -- --nova   # roda até a 1ª pausa e
   pergunta; [s] sai e deixa pausado
   pnpm -F @mentoria/curador m07:edicao             # continua de onde parou
   ```

   Grava `saidas/m07-relatorio.md` com as decisões e as métricas.

Responda (vamos discutir na Aula 8):

- Quantas pausas, qual a taxa de intervenção e de override? Você mudou alguma decisão da máquina?
- Algum item que parou não precisava parar? Algum que passou direto deveria ter parado?
- Que parte você deixaria 100% automática numa versão agendada (Aula 8)?

**Desafio extra:** troque o terminal por outra "tela" (uma página local, um bot de chat): o grafo não muda, só quem chama `Command({ resume })`.

# Para revisar

## Perguntas para discussão (15 min)

1. No seu trabalho, que decisão você deixaria 100% automática, qual pediria aprovação e qual nunca deixaria com uma máquina? Por quê?
2. Se a pessoa que revisa aprova 99% dos itens sem mudar nada, o que isso indica? Como você descobriria se ela está realmente olhando?
3. No curador, a pausa da edição inteira vale o tempo de alguém toda semana? O que precisaria acontecer para tirá-la?
4. Quem deveria definir os limites da política de reembolso (R$ 100, R$ 2.000, confiança 0,9)? Com que dados?
5. Uma pausa pode durar dias. O que acontece com o mundo enquanto isso (a notícia envelhece, o cliente desiste)? Como lidar?

## Armadilhas comuns

- **Efeito externo antes do `interrupt()`:** ao retomar, o nó roda de novo, e o e-mail (ou o pagamento) sai duas vezes.
- **Chamada cara de LLM antes do `interrupt()`:** é repetida a cada retomada, inclusive quando você responde outras pausas em paralelo.
- **`interrupt()` dentro de `try/catch`:** o `catch` engole a pausa.
- **Aceitar a resposta humana sem validar:** um "sim" solto ou um valor absurdo vira aprovação.
- **Pedir aprovação de tudo:** a revisão vira carimbo (viés de automação) e o processo fica lento.
- **Pausa sem contexto:** "aprova?" sem o motivo da pausa, a fonte e a confiança.
- **Limiar de automação no chute, para sempre:** recalibre com o histórico de decisões.
- **Não registrar as decisões humanas:** sem elas, não há métricas nem dados para soltar a automação.
- **Pausar sem checkpointer persistente:** a pausa some se o processo morrer.